

Aim – Using the Iris Data (<https://www.kaggle.com/datasets/saurabh00007/iris.csv>), perform the following tasks:

- i. Read the first 8 rows of the dataset.
- ii. Display the column names of the Iris dataset.
- iii. Fill any missing data with the mean value of the respective column.
- iv. Remove rows that contain any missing values.
- v. Calculate and display the mean, minimum, and maximum values of the Sepal length column.

LO6

17 Using the Cars Data (<https://www.kaggle.com/datasets/nameeerafatima/toyotacsv>) perform the following tasks:

- i. Create a scatter plot between the Age and Price of the cars to illustrate how the price decreases as the age of the car increases.
- ii. Generate a histogram to show the frequency distribution of kilometres driven by the cars.
- iii. Produce a bar plot to display the distribution of cars by fuel type.
- iv. Create a pie chart to represent the percentage distribution of cars based on fuel types.
- v. Draw a box plot to visualize the distribution of car prices across different fuel types.

Theory –

This program demonstrates basic data analysis and visualization using Python libraries like Pandas and Matplotlib. First, the Iris dataset is loaded to explore data preprocessing techniques such as handling missing values. The dataset is inspected by displaying the first few rows and column names. Missing values are managed using two approaches: filling numerical columns with mean values and removing rows containing null values. Statistical analysis is then performed on the “SepalLengthCm” column to calculate mean, minimum, and maximum values.

In the second part, the Toyota dataset is used for visual analysis. Various plots are created to understand relationships and distributions within the data. A scatter plot shows how car price varies with age, while a histogram displays the distribution of kilometers driven. Bar and pie charts illustrate the frequency and percentage of different fuel types. Finally, a boxplot compares price distributions across fuel types, helping identify variations and outliers effectively.

A1.

```
import pandas as pd
```

```
df = pd.read_csv('iris.csv')
```

```

print("First 8 rows of the dataset:")
print(df.head(8))

print("\nColumn names:")
print(df.columns.tolist())

df_filled = df.fillna(df.mean(numeric_only=True))

df_cleaned = df.dropna()

sepal_length = df_cleaned['SepalLengthCm']

print("\nSepal Length Statistics:")
print(f"Mean: {sepal_length.mean()}")
print(f"Minimum: {sepal_length.min()}")
print(f"Maximum: {sepal_length.max()}")

```

```

First 8 rows of the dataset:
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  Species
0   1             5.1             3.5             1.4             0.2  Iris-setosa
1   2             4.9             3.0             1.4             0.2  Iris-setosa
2   3             4.7             3.2             1.3             0.2  Iris-setosa
3   4             4.6             3.1             1.5             0.2  Iris-setosa
4   5             5.0             3.6             1.4             0.2  Iris-setosa
5   6             5.4             3.9             1.7             0.4  Iris-setosa
6   7             4.6             3.4             1.4             0.3  Iris-setosa
7   8             5.0             3.4             1.5             0.2  Iris-setosa

Column names:
['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm', 'Species']

Sepal Length Statistics:
Mean: 5.843333333333334
Minimum: 4.3
Maximum: 7.9

```

A2.

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
df = pd.read_csv('Toyota.csv')
```

```
plt.figure(figsize=(10, 6))
plt.scatter(df['Age'], df['Price'], alpha=0.6, color='blue')
plt.xlabel('Age (years)')
plt.ylabel('Price')
plt.title('Car Price vs Age')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(10, 6))
plt.hist(df['KM'], bins=30, color='green', edgecolor='black')
plt.xlabel('Kilometres Driven')
plt.ylabel('Frequency')
plt.title('Distribution of Kilometres Driven')
plt.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(10, 6))
fuel_counts = df['FuelType'].value_counts()
plt.bar(fuel_counts.index, fuel_counts.values, color='orange', edgecolor='black')
plt.xlabel('Fuel Type')
plt.ylabel('Number of Cars')
plt.title('Distribution of Cars by Fuel Type')
plt.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(8, 8))
fuel_counts = df['FuelType'].value_counts()
plt.pie(fuel_counts.values, labels=fuel_counts.index, autopct='%1.1f%%', startangle=90)
plt.title('Percentage Distribution of Cars by Fuel Type')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
plt.figure(figsize=(10, 6))
```

```
df.boxplot(column='Price', by='FuelType', figsize=(10, 6))
```

```
plt.xlabel('Fuel Type')
```

```
plt.ylabel('Price')
```

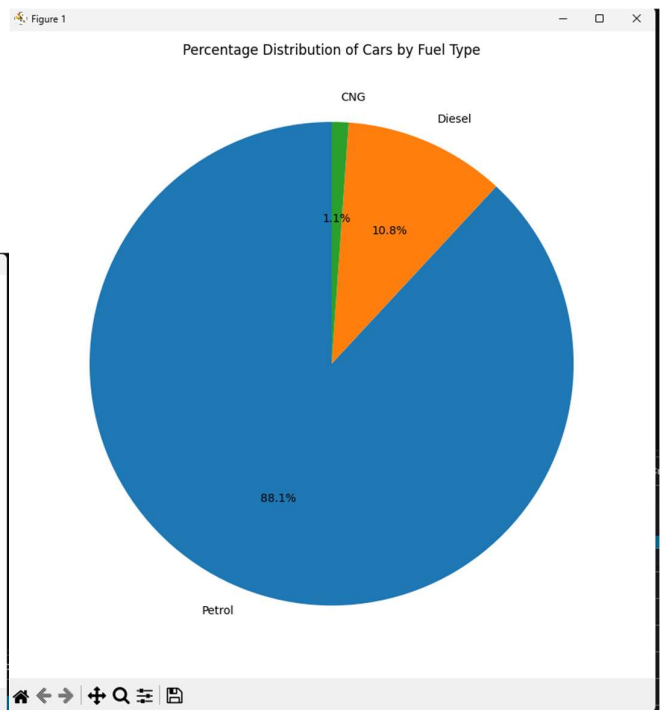
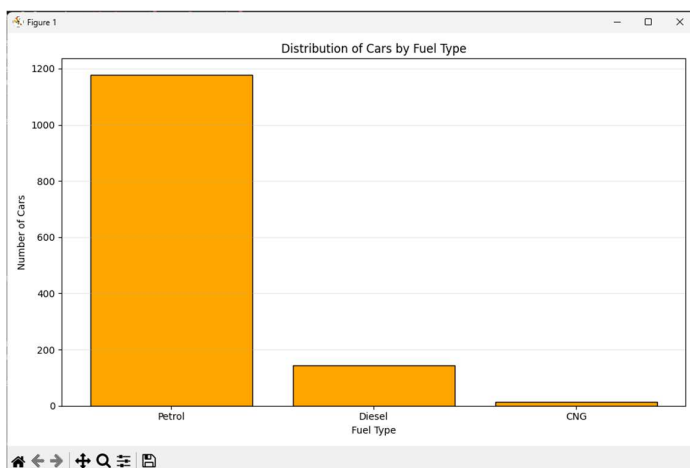
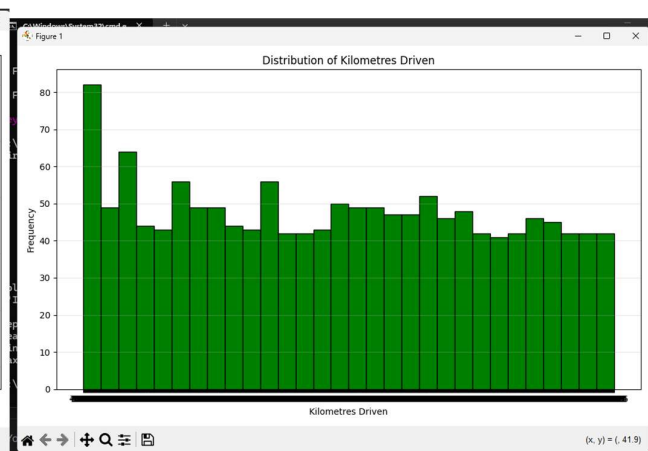
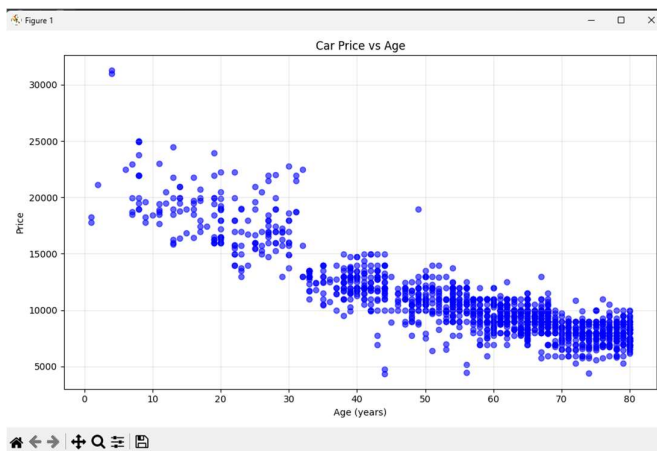
```
plt.title('Distribution of Car Prices by Fuel Type')
```

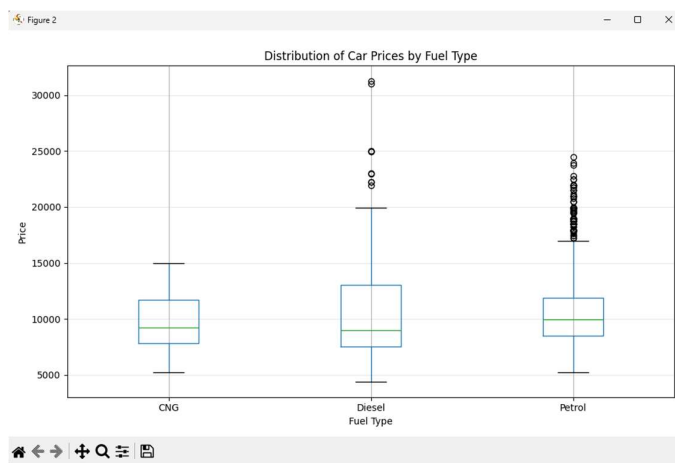
```
plt.suptitle("") # Remove the default title
```

```
plt.grid(True, alpha=0.3, axis='y')
```

```
plt.tight_layout()
```

```
plt.show()
```





Conclusion

The program effectively combines data cleaning, statistical analysis, and visualization techniques. It highlights how datasets can be explored and interpreted using Python. Visual tools make patterns and relationships clearer, while preprocessing ensures accuracy, making the overall analysis more meaningful and reliable for decision-making.